1: Introduction

String is the **basic data type** in Redis.

- Stores **text**, **numbers**, or **binary data**
- Max size: **512 MB**
- Fast read/write because data is stored in memory

2. Starting Redis Command Line

```
C:\Users\sugas>redis-cli
127.0.0.1:6379>
```

If Redis is running, you will see:

127.0.0.1:6379>

3. SET and GET (Store and Retrieve)

`SET` stores a value for a key
`GET` retrieves the stored value

```
C:\Users\sugas>redis-cli
127.0.0.1:6379> set name sugan
OK
127.0.0.1:6379> get name
"sugan"
127.0.0.1:6379>
```

4. Storing Numbers as Strings

Redis stores numbers as strings but allows arithmetic operations.

```
127.0.0.1:6379> set age 22
OK
127.0.0.1:6379> get age
"22"
127.0.0.1:6379>
```

5. Increment and Decrement

Used for counters such as page views, likes, login count.

```
127.0.0.1:6379> INCR age
(integer) 23
127.0.0.1:6379> DECR age
(integer) 22
127.0.0.1:6379>
```

```
127.0.0.1:6379> INCRBY age 5
(integer) 27
127.0.0.1:6379> DECRBY age 2
(integer) 25
127.0.0.1:6379>
```

6. Student Marks Example

Keys can be structured using : for readability.

```
127.0.0.1:6379> set student:101:marks 85
OK
127.0.0.1:6379> get student:101:marks
"85"
127.0.0.1:6379>
```

7. APPEND Command

Adds text to an existing string.

```
127.0.0.1:6379> set student:101:marks 85
OK
127.0.0.1:6379> get student:101:marks
"85"
127.0.0.1:6379> set msg "hello"
OK
127.0.0.1:6379> append msg " students"
(integer) 14
127.0.0.1:6379> get msg
"hello students"
127.0.0.1:6379>
```

8. Multiple Strings (MSET and MGET)

Store and retrieve multiple key–value pairs at once.

```
127.0.0.1:6379> MSET s1 "Math" s2 "Sci" s3 "Eng"
OK
127.0.0.1:6379> MGET s1 s2 s3
1) "Math"
2) "Sci"
3) "Eng"
```

9. Check if Key Exists

Returns `1` if key exists, `0` otherwise.

```
127.0.0.1:6379> EXISTS name
(integer) 1
127.0.0.1:6379>
```

10. Getrange

`GETRANGE` is used to **retrieve a substring** from a Redis string value.

It returns characters from a string **by index**.

```
127.0.0.1:6379> GET msg
"hello students"
127.0.0.1:6379> GETRANGE msg 0 5
"hello "
127.0.0.1:6379> GETRANGE msg 6 13
"students"
127.0.0.1:6379> |
```

11. Using Negative Index

`-1` refers to the **last character**.

```
127.0.0.1:6379> GETRANGE msg 6 -1
"students"
127.0.0.1:6379> GETRANGE msg 6 -2
"student"
127.0.0.1:6379> GETRANGE msg 6 -3
"studen"
127.0.0.1:6379> |
```

12. GETRANGE on Numbers

Numbers are stored as strings, so substring operations are allowed.

```
127.0.0.1:6379> set regno 20251234
OK
127.0.0.1:6379> GETRANGE regno 0 3
"2025"
127.0.0.1:6379> |
```

13. Password Masking

Display only the first 2 characters.

```
127.0.0.1:6379> SET password "redis123"
OK
127.0.0.1:6379> GETRANGE password 0 1
"re"
127.0.0.1:6379>
```

14. OTP Partial Display

Show only the last 2 digits of OTP.

```
127.0.0.1:6379> SET password "redis123"
OK
127.0.0.1:6379> GETRANGE password 0 1
"re"
127.0.0.1:6379> SET otp 98765
OK
127.0.0.1:6379> GETRANGE otp -2 -1
"65"
```

15. SETRANGE

`SETRANGE` overwrites **part of an existing Redis string** starting at a specified **offset (index)**.

It **modifies the stored value**, unlike `GETRANGE`.

SETRANGE key offset value

- `offset` → zero-based index where replacement starts
- `value` → string to be written
- If the key does not exist, Redis treats it as an empty string
- Returns the **length of the string after modification**

```
127.0.0.1:6379> SET message "Hello World"
OK
127.0.0.1:6379> SETRANGE message 6 "Redis"
(integer) 11
127.0.0.1:6379> GET message
"Hello Redis"
127.0.0.1:6379>
```

16. SETRANGE on an Empty Key

If the key does not exist, Redis fills missing characters with `\x00`.

```
127.0.0.1:6379> SETRANGE newkey 3 "Hi"
(integer) 5
127.0.0.1:6379> GET newkey
"\x00\x00\x00Hi"
```

17. Modify Numeric String

Numbers are strings in Redis, so characters can be replaced.

```
127.0.0.1:6379> SET year 2025
OK
127.0.0.1:6379> SETRANGE year 3 "267"
(integer) 6
127.0.0.1:6379> GET year
"202267"
```

18. Real-Time Use Case – Version Update

Update part of a version number.

```
127.0.0.1:6379> SET app:version "v1.0.0"
OK
127.0.0.1:6379> SETRANGE app:version 3 "1"
(integer) 6
127.0.0.1:6379> GET app:version
"v1.1.0"
127.0.0.1:6379>
```

19. Masking sensitive data

Replace part of a password with `****`.

```
127.0.0.1:6379> SET password "redis123"
OK
127.0.0.1:6379> SETRANGE password 2 "*****"
(integer) 8
127.0.0.1:6379> GET password
"re*****3"
127.0.0.1:6379>
```

20. Overwrite Existing Key

Since the key already exists, Redis will **not update it**.

```
127.0.0.1:6379> GET name
(nil)
127.0.0.1:6379> SET name1 "Ren" NX
OK
127.0.0.1:6379> GET name1
"Ren"
```

21. Distributed Lock (Real-Time Use Case)

Only one process can acquire the lock.

```
127.0.0.1:6379> SET lock:payment "LOCKED" NX EX 10
OK
127.0.0.1:6379> GET lock:payment
"LOCKED"
127.0.0.1:6379>
```

Important Points

- `NX` works **only with `SET`**
- Prevents accidental overwrites
- Commonly used in **locking mechanisms**
- Returns `(nil)` if key exists

22: XX

`XX` ensures that a value is **updated only for an existing key** and **never creates a new key**.

```
127.0.0.1:6379> SET name2 "Lee"
OK
127.0.0.1:6379> SET name2 "May" XX
OK
127.0.0.1:6379> GET name1
"Ren"
127.0.0.1:6379> GET name2
"May"
```

23. Key Does NOT Exist

```
127.0.0.1:6379> SET city "Chennai" XX
(nil)
127.0.0.1:6379> GET city
(nil)
```

24: Safe Update (Real-Time Use Case)

Avoid accidental creation of keys while updating.

25. SETS

In **Redis**, a **Set** is an **unordered collection of unique elements**.

No duplicates, no order

Key Characteristics

- Elements are **unique**
- **Unordered** (no index)
- Very fast operations
- Supports **mathematical set operations** (union, intersection, difference)

Command	Purpose
SADD	Add member(s)
SMEMBERS	Get all members
SISMEMBER	Check membership
SREM	Remove member
SCARD	Count members

Command	Purpose
SUNION	Union of sets
SINTER	Intersection
SDIFF	Difference

1. SADD – Add Elements to a Set
   Adds one or more elements to a set.

```
127.0.0.1:6379> SADD courses "Math" "Science" "Eng"
(integer) 3
127.0.0.1:6379> SADD courses "Math"
(integer) 0
127.0.0.1:6379>
```

2. SMEMBERS – View All Elements
   Returns all unique elements (order not guaranteed).

```
127.0.0.1:6379> SMEMBERS courses
1) "Eng"
2) "Math"
3) "Science"
127.0.0.1:6379>
```

3. SISMEMBER – Check Membership
   Checks whether a value exists in the set.

```
127.0.0.1:6379> SISMEMBER courses "Math"
(integer) 1
127.0.0.1:6379>
```

4. SCARD – Count Elements
   Returns number of elements in the set.

```
127.0.0.1:6379> SCARD courses
(integer) 3
127.0.0.1:6379>
```

5. SREM – Remove an Element

```
127.0.0.1:6379> SREM courses "Eng"
(integer) 1
127.0.0.1:6379> SMEMBERS courses
1) "Math"
2) "Science"
127.0.0.1:6379>
```

6. Real-Time Example: Student Registration

```
127.0.0.1:6379> SADD registered_studs 101 102 103
(integer) 3
127.0.0.1:6379> SADD registered_studs 101
(integer) 0
127.0.0.1:6379>
```

7. SUNION – Union of Sets

Returns all unique elements from both sets.

```
127.0.0.1:6379> SADD classA 1 2 3
(integer) 3
127.0.0.1:6379> SADD classB 3 4 5
(integer) 3
127.0.0.1:6379> SUNION classA classB
1) "1"
2) "2"
3) "3"
4) "4"
5) "5"
127.0.0.1:6379>
```

8. SINTER – Intersection of Sets

```
127.0.0.1:6379> SINTER classA classB
1) "3"
127.0.0.1:6379>
```

9. SDIFF – Difference of Sets

```
127.0.0.1:6379> SINTER classA classB
1) "3"
127.0.0.1:6379> SDIFF classA classB
1) "1"
2) "2"
127.0.0.1:6379>
```

Feature	SET	LIST
Order	No	Yes
Duplicates	No	Yes
Indexing	No	Yes

26 LISTS

In **Redis**, a **List** is an **ordered collection of elements** stored as a **linked list**.

Order is maintained, duplicates are allowed.

Key Characteristics

- **Ordered** (index-based)
- **Duplicates allowed**
- Fast insertion/deletion at both ends
- Supports queue and stack operations

Command	Purpose
LPUSH	Insert at left
RPUSH	Insert at right
LPOP	Remove from left
RPOP	Remove from right
LRANGE	Get elements
LLEN	Length of list
LINDEX	Get by index

1. LPUSH – Insert at Beginning

```
127.0.0.1:6379> LPUSH subjects "HIVE"
(integer) 5
127.0.0.1:6379> LRANGE subjects 0 -1
1) "HIVE"
2) "Bigdata"
3) "NoSQL"
4) "BigData"
5) "NoSQL"
127.0.0.1:6379>
```

2. RPUSH – Insert at End

```
127.0.0.1:6379> RPUSH subjects "Eng"
(integer) 6
127.0.0.1:6379> LRANGE subjects 0 -1
1) "HIVE"
2) "Bigdata"
3) "NoSQL"
4) "BigData"
5) "NoSQL"
6) "Eng"
127.0.0.1:6379>
```

3. LRANGE – View List Elements

```
127.0.0.1:6379> LRANGE subjects 0 -1
1) "HIVE"
2) "Bigdata"
3) "NoSQL"
4) "BigData"
5) "NoSQL"
6) "Eng"
```

4. LPOP – Remove from Left

```
127.0.0.1:6379> LPOP subjects
"HIVE"
127.0.0.1:6379>
```

5. RPOP – Remove from Right

```
127.0.0.1:6379> LPOP subjects
"HIVE"
127.0.0.1:6379> RPOP subjects
"Eng"
```

6. LLEN – Length of List

```
127.0.0.1:6379> LLEN subjects
(integer) 4
127.0.0.1:6379>
```

7. LINDEX – Access by Index

```
127.0.0.1:6379> LINDEX subjects 0
"Bigdata"
127.0.0.1:6379>
```

8. Real-Time Example: Queue (FIFO)

```
127.0.0.1:6379> RPUSH tasks "task1"
(integer) 1
127.0.0.1:6379> RPUSH tasks "task2'
Invalid argument(s)
127.0.0.1:6379> RPUSH tasks "task2"
(integer) 2
127.0.0.1:6379> LPOP tasks
"task1"
127.0.0.1:6379>
```

9. Real-Time Example: Stack (LIFO)

```
127.0.0.1:6379> LPOP tasks
"task1"
127.0.0.1:6379> LPUSH stack "A"
(integer) 1
127.0.0.1:6379> LPUSH stack "B"
(integer) 2
127.0.0.1:6379> LPOP stack
"B"
127.0.0.1:6379>
```

27. Delete a String

Removes a key and its value.

```
127.0.0.1:6379> EXISTS name
(integer) 1
127.0.0.1:6379> DEL name
(integer) 1
127.0.0.1:6379> GET name
(nil)
```

28. Set Expiry Time (TTL)

Automatically deletes the key after a given time (seconds).

```
127.0.0.1:6379> SET otp 123456 EX 20
OK
127.0.0.1:6379> GET otp
"123456"
127.0.0.1:6379> GET otp
(nil)
127.0.0.1:6379>
```

29. Real-Time Example: Page View Counter

Each visit increases the count.

```
127.0.0.1:6379> INCR page:home:views
(integer) 1
127.0.0.1:6379> INCR page:home:views
(integer) 2
127.0.0.1:6379> GET page:home:views
"2"
127.0.0.1:6379>
```

30. Hashes

In **Redis**, a **Hash** is a data type used to store **multiple field–value pairs inside a single key**.

Think of a Redis Hash like a **row in a table** or a **JSON object**.

Why Use Hashes?

- Group related information together
- Save memory
- Faster access to individual fields
- Ideal for storing **objects**

Basic Structure

key → { field1: value1, field2: value2, field3: value3 }

1. HSET – Store Fields in a Hash

```
127.0.0.1:6379> HSET student:101 name "Ben" age 21 marks 85
(integer) 3
```

2. HGET – Get a Single Field Value

```
127.0.0.1:6379> HGET student:101 name
"Ben"
127.0.0.1:6379>
```

3. HGETALL – Get All Fields and Values

```
127.0.0.1:6379> HGETALL student:101
1) "name"
2) "Ben"
3) "age"
4) "21"
5) "marks"
6) "85"
127.0.0.1:6379>
```

4. HMGET – Get Multiple Fields

```
127.0.0.1:6379> HMGET student:101 name marks
1) "Ben"
2) "85"
127.0.0.1:6379>
```

5. HEXISTS – Check if Field Exists
   Returns `1` if field exists, else `0`.

```
127.0.0.1:6379> HEXISTS student:101 age
(integer) 1
127.0.0.1:6379>
```

6. HDEL – Delete a Field

```
127.0.0.1:6379> HGETALL student:101
1) "name"
2) "Ben"
3) "marks"
4) "85"
127.0.0.1:6379>
```

7. HLEN – Count Fields

```
127.0.0.1:6379> HLEN student:101
(integer) 2
127.0.0.1:6379>
```

8. HINCRBY – Increment Numeric Field

```
127.0.0.1:6379> HINCRBY student:101 marks 5
(integer) 90
127.0.0.1:6379> HGETALL student:101
1) "name"
2) "Ben"
3) "marks"
4) "90"
127.0.0.1:6379>
```

9. Real-Time Example: User Profile

```
127.0.0.1:6379> HSET user:501 username "Ben" email "ben@gmai.com"
(integer) 2
127.0.0.1:6379> HGETALL user:501
1) "username"
2) "Ben"
3) "email"
4) "ben@gmai.com"
127.0.0.1:6379>
```

HASH vs STRING

Feature	STRING	HASH
Stores	Single value	Field–value pairs
Structure	Simple	Structured
Use case	Counter, OTP	User profile, object